МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных технологий

Отчет по практической работе №9

по дисциплине «Разработка кроссплатформенных мобильных приложений»

Выполнил:
Студент группы ИКБО-07-22 Савин А.С.

Проверил:
Старший преподаватель кафедры МОСИТ Шешуков Л.С.

Москва 2025 г.

Цели практической работы:

- Ознакомиться с задачами хранения состояния;
- Произвести знакомство с Bloc;
- Произвести знакомство с Riverpod;
- Произвести знакомство с MobX;
- Выполнение практической работы №9

1. Задача хранения состояния

Управление состоянием мобильных приложений – это фундаментальная концепция всех мобильных фреймворков для разработки приложений, и Flutter не является исключением. Управление состоянием – это то, как приложение может управлять данными, которые оно использует, и обновлять их. Во Flutter существует несколько техник и инструментов, которые можно использовать для управления состоянием, и выбор наиболее подходящего из них часто зависит от сложности и требований приложения, которое создается.

Хранение состояний экрана играет ключевую роль в разработке, но сопряжено с рядом проблем. Среди проблем масштабирования можно выделить ограниченную переиспользуемость компонентов, возникающую из-за зависимости виджетов от конкретного состояния экрана, а также нарушение принципа разделения ответственности, когда экран одновременно управляет отображением и состоянием. Проблемы производительности включают неэффективное использование памяти, поскольку состояние хранится в каждом экземпляре экрана, и неоптимальные обновления UI, когда изменения состояния в одном месте требуют синхронизации с другими частями приложения. К проблемам архитектуры относятся сложность тестирования, так как состояние тесно связано с UI-логикой, и сложность отладки, поскольку состояние может изменяться в разных местах кода, что затрудняет отслеживание всех возможных путей изменений.

2. Произвести знакомство с Bloc

BLoC — это акроним от «Business Logic Component» (компонент бизнес-логики). Как следует из названия, это класс, отделяющий бизнес-логику приложения от пользовательского интерфейса. Такой компонент содержит код, который можно повторно использовать где угодно: в другом модуле, на другой платформе, в другом приложении. Вся работа строится на основании состояний двух каналов данных: Event и State stream.

Существуют два вида Bloc: Stream Bloc, где для изменения состояния необходимо отправить специальный event и по его итогу получить некий State, и Cubit Bloc, где events отсутствует, и мы напрямую обращаемся в Cubit, который в свою очередь генерирует новый State. В Cubit нет Event Stream, его логику берут на себя чистые функции контейнера.

Основные части хранилища BLoC включают:

- Event Stream — поток событий, который позволяет UI-компонентам отправлять команды в BLoC;
- State Stream — выходной поток, который передает изменения состояния из BLoC в UI;
- BLoC container — компонент, который обеспечивает доступ к BLoC для UI-компонентов;
- BlocObserver — механизм для отслеживания изменений состояния и событий в BLoC;
- emit — механизм для обновления состояния в BLoC.

2.1 Подключение пакетов

Для реализации работы с Bloc в проект требуется в файл pubspec.yaml добавить пакеты bloc версии 9.1.0 и flutter_bloc версии 9.1.1. Данные версии были выбраны, так как они является самыми актуальными на момент выполнения данной практической работы, как показано на рисунке 1.

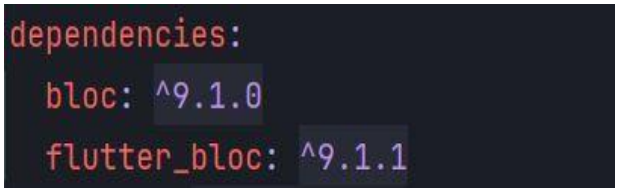


Рисунок 1 — Добавление пакетов для работы с Bloc

2.2 Реализация общего слушателя

Первое, что будет рассмотрено, это как создать объект BlocObserver, который позволяет наблюдать за всеми изменениями состояния в приложении. Создадим файл counter_observer.dart с содержимым, проиллюстрированным на рисунке 2.



Рисунок 2 — Класс CounterObserver

2.3 Подготовка Widget-а приложения

Далее требуется доработать содержимое функции main. По итогу функция должена иметь содержимое, показанное на рисунке 3.



Рисунок 3 — Функция main

Widget приложение CounterApp также требуется изменить. В качестве итогового результата требуется получить Widget с указанием основной страницы, как показано на рисунке 4.

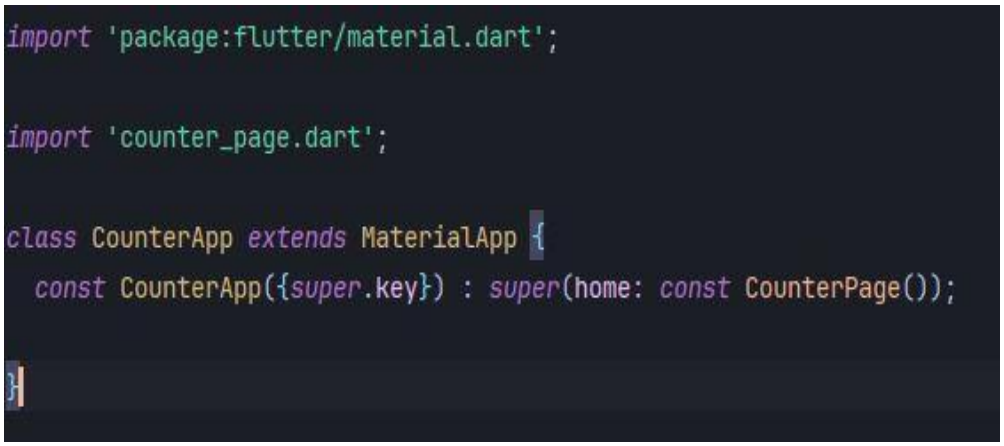


Рисунок 4 — CounterApp

2.4 Реализация страницы счётчика

После того, как Widget приложения готов к работе, необходимо создать

страницу со счетчиком. Для этого создадим класс CounterPage с содержимым, указанным на рисунке 5.

```dart
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter/material.dart';
import 'counter_cubit.dart';
import 'counter_view.dart';

class CounterPage extends StatelessWidget{

  const CounterPage({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
        create: (_) => CounterCubit(),
        child: const CounterView()
    );
  }
}
```

Рисунок 5 — CounterPage

2.5 Реализация Cubit класса

После того, как страница создана и к ней подвязаны все необходимые классы, нужно создать класс-делегат – Cubit, показанным на рисунке 6.

```dart
import 'package:bloc/bloc.dart';

class CounterCubit extends Cubit<int>{

  CounterCubit(): super(0);

  void increment() => emit(state + 1);

  void decrement() => emit(state - 1);
}
```

Рисунок 6 — CounterCubit

Класс CounterCubit выступает в приложении как класс-делегат. В его задачи закладывается хранение данных для отображения на экране и предоставление методов для изменения этих данных.

Для хранения данных родительскому классу Cubit в значение дженерик класса указан тип хранимых данных – int, а в конструкторе класса CounterCubit в родительском конструкторе задается стартовое значение, равное 0. В

качестве методов изменения данных класс CounterCubit предоставит два метода:

- increment – добавляет 1 к текущему состоянию
- decrement – вычитает 1 из текущего состояния

2.6 Реализация представления данных Bloc

Когда сам экран и класс-делегат созданы, можно приступать к реализации класса представления данных. Создадим класс CounterView, с содержимым, указанным на рисунке 7. Класс CounterView отвечает за рендеринг текущего счетчика и рендеринг двух FloatingActionButtons для увеличения/уменьшения счетчика.

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'counter_cubit.dart';

class CounterView extends StatelessWidget {

  const CounterView({super.key});

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;
    return Scaffold(
      body: Center(
        child: BlocBuilder<CounterCubit, int>(
          builder: (context, state) {
            return Text(
              '$state',
              style: textTheme.displayMedium,
            ); Text
          },
        ), BlocBuilder
      ), Center
      floatingActionButton: Column(
        mainAxisAlignment: MainAxisAlignment.end,
        crossAxisAlignment: CrossAxisAlignment.end,
        children: <Widget>[
          FloatingActionButton(
            key: const Key('counterView_increment_floatingActionButton'),
            child: const Icon(Icons.add),
            onPressed: () => context.read<CounterCubit>().increment(),
          ), FloatingActionButton
          const SizedBox(height: 8),
          FloatingActionButton(
            key: const Key('counterView_decrement_floatingActionButton'),
            child: const Icon(Icons.remove),
            onPressed: () => context.read<CounterCubit>().decrement(),
          ), FloatingActionButton
        ], <Widget>[]
      ), Column
    ); Scaffold
  }
}
```

Рисунок 7 — CounterView

В данной реализации BlocBuilder используется для обновления Text Widget-a при каждом изменении состояния в CounterCubit. Для связки

CouterView с CounterCubit в BlocBuilder в качестве первого дженерик типа устанавливается интересуемый класс-делегат, а в качестве второго типа указывается тип данных, который будет приходить в качестве аргумента state метода builder. Для изменения данных в классе-делегате используется context.read(). Эта команда используется для поиска ближайшего CounterCubit экземпляра и получению к нему доступа.

3. Riverpod

По своей сути Riverpod – это библиотека управления состоянием для Flutter, которая призвана улучшить другую имеющуюся библиотеку – Provider. Riverpod обеспечивает более безопасный, тестируемый и гибкий способ управления состоянием и его совместного использования в приложении, чем его более старая версия – Provider.

Provider — это пакет для управления состоянием в Flutter, который упрощает передачу данных между виджетами. Он построен на основе InheritedWidget и предоставляет удобный интерфейс для работы с состоянием приложения. Принцип работы ничем не отличается от работы с Inherited Widget. Отличными чертами являются методы вызова provider: метод read отвечает за разовое прочтение данных, метод watch отвечает за обновление экранной формы при изменении данных.

Основные части хранилища Provider:

- Provider container представляет собой обёртку вокруг InheritedWidget, которая позволяет передавать любой объект вниз по дереву виджетов;
- ChangeNotifier Provider является специализированной версией базового Provider;
- Listenable Provider работает по похожему принципу с ChangeNotifier Provider, но он более универсален;
- Stream Provider предназначен для работы с потоками данных (Stream);

- Multi Provider — это инструмент для организации нескольких провайдеров в одном месте.

Riverpod использует неизменяемые объекты-провайдеры и концепцию ScopedProviders. Это как система с отдельными зонами видимости, где каждый провайдер существует независимо от дерева виджетов. В Provider работа происходит напрямую с BuildContext и используются различные провайдеры (ChangeNotifierProvider, ValueNotifierProvider). Это похоже на прямую дорогу от виджета к данным.

Основные части хранилища Riverpod:

- StateNotifier Provider — основной компонент для управления состоянием, он объединяет концепцию StateNotifier и Provider, обеспечивая автоматическое обновление UI при изменении состояния;
- Provider Scope — корневой виджет, который оборачивает всё приложение и управляет всеми провайдерами, он является точкой входа для всей системы управления состоянием;
- Scoped Providers позволяют ограничивать область видимости провайдеров определённой частью дерева виджетов, это особенно полезно для больших приложений с сложной структурой.

3.1 Подключение пакетов

Для реализации работы с Riverpod в проект требуется в файл pubspec.yaml добавить пакеты flutter_riverpod версии 3.0.3, riverpod_annotation версии 3.0.3, riverpod_generator версии 3.0.3, riverpod_lint версии 3.0.3. Данные версии были выбраны, так как они является самыми актуальными на момент выполнения данной практической работы. Кроме того, были подключены пакеты build_runner версии 2.7.1 и custom_lint версии 0.8.0, версии которых были выбраны, так как являются совместимыми с версиями Dart и Flutter используемыми в проекте, как показано на рисунке 8.

```
dependencies:
  flutter_riverpod: ^3.0.3
  riverpod_annotation: ^3.0.3
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8

dev_dependencies:
  build_runner: ^2.7.1
  custom_lint: ^0.8.0
  riverpod_generator: ^3.0.3
  riverpod_lint: ^3.0.3
  flutter_test:
    sdk: flutter
```

Рисунок 8 — Подключение пакетов для работы с Riverpod

3.2 Подготовка Widget-а приложения

Далее требуется доработать содержимое lib/main.dart. По итогу файл main.dart должен иметь содержимое, показанное на рисунке 9. В нем Widget приложения оборачивается в Widget ProviderScope, который направлен на хранение всех классов-делегатов в рамках этой области видимости.

```
void main() {
  runApp(
    ProviderScope(child: CounterApp())
  );
}
```

Рисунок 9 — Файл main.dart

Виджет приложения изменять нет необходимости. Он останется таким же, как в стартовом проекте, как показано на рисунке 10.

```
class CounterApp extends MaterialApp {
  const CounterApp({super.key}) : super(home: const CounterPage());

}
```

Рисунок 10 – Виджет приложения

3.3 Реализация страницы счётчика

Виджет страницы представлен на рисунке 11.

```dart
class CounterPage extends StatelessWidget{

  const CounterPage({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: CounterView());
  }
}
```

Рисунок 11 — Виджет страницы

После того, как виджет страницы готов к работе, необходимо создать представление со счетчиком. Для этого создадим класс CounterView с содержимым, указанным на рисунке 12.

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'counter.dart';

class CounterView extends ConsumerWidget {
  const CounterView({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Counter example'),
      ), AppBar
      body: Center(
        child: Text('${ref.watch(counterProvider)}'),
      ), Center
      floatingActionButton: FloatingActionButton(
          onPressed: () => ref.read(counterProvider.notifier).increment(),
      child: const Icon(Icons.add),
    ), FloatingActionButton
    ); Scaffold
  }
}
```

Рисунок 12 — Виджет представления

В реализации представления в рамках использования пакета Riverpod наблюдатель данных наследуется не от Stateless или Stateful Widget, а от CinsumerWidget класса. Это наследование меняет сигнатуру стандартного метода build, добавляя ему в качестве дополнительного аргумента еще WidgetRef объект. WidgetRef объект используется для настройки получения и прослушивания данных в классе делегате. Для простого получения данных из делегата необходимо использовать метод read, а для наблюдения за данными и автоматическом обновлении экрана при их изменении, нужно использовать метод watch.

3.4 Реализация класса-делегата

После того, как представления создано, нужно создать класс делегат в файле lib/counter.dart как показано на рисунке 13.

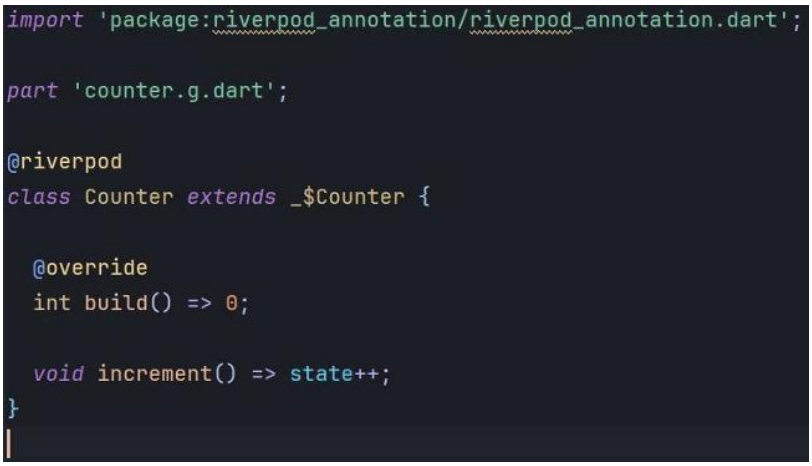


Рисунок 13 – Класс-делегат

Для указания, что класс Counter является классом-делегатом, требуется установить на него обязательную аннотацию - @riverpod. При указании данной аннотации класс должен иметь метод build, который должен возвращать наблюдаемые данные в классе-делегате. В качестве методов изменения данных класс Counter предоставляет метод – increment, который добавляет 1 к текущему состоянию. Класс наследуется от _$Counter, определённого в сгенерированном файле counter.g.dart.

Чтобы сгенерировать файл counter.g.dart, необходимо ввести команду flutter pub run build_runner build --delete-conflicting-outputs, как показано на рисунке 14.

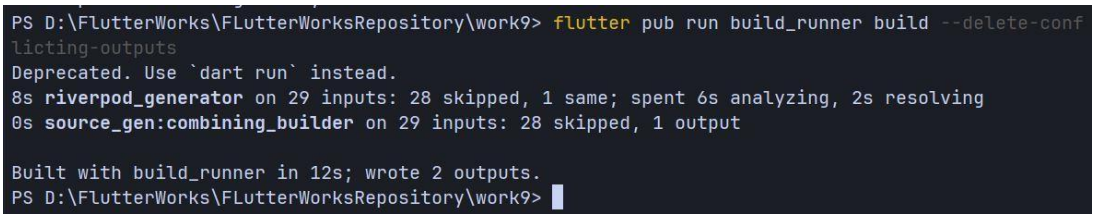


Рисунок 14 — Запуск команды

4. Mobx

MobX – это библиотека управления состоянием, в которой применяется концепция реактивного программирования. Она разделяет состояние и пользовательский интерфейс приложения и автоматически обновляет интерфейс при изменении состояния.

В реактивном подходе MobX ключевыми элементами являются: Observable — данные, за изменениями которых следит система; Actions — операции, изменяющие состояние данных; Computed — значения, автоматически обновляющиеся при изменении зависимостей.

Основные части хранилища MobX:

- Observable выступает в роли системы мониторинга состояния, когда мы помечаем данные как наблюдаемые, MobX автоматически создает граф зависимостей между ними;
- Readonly является аналогом Observable, однако запрещает изменение состояния значения извне хранилища;
- Computed представляет собой механизм для создания производных значений, которые автоматически пересчитываются при изменении исходных данных;
- Action — это специальные функции, предназначенные для модификации состояния, они играют роль защитного слоя, гарантирующего, что все изменения будут произведены корректно и атомарно;
- Reaction — это механизм, который позволяет выполнять определенные действия при изменении состояния, он работает как система триггеров, автоматически реагирующая на изменения данных и выполняющая заданные операции.

4.1 Подключение пакетов

Для реализации работы с MobX в проект требуется в файл pubspec.yaml добавить пакеты mobx версии 2.5.0, flutter_mobx версии 2.3.0, build_runner версии 2.7.1 mobx_codegen версии 2.7.4, как показано на рисунках 15–16. Данные версии были выбраны, так как они является самыми актуальными на момент выполнения данной практической работы.

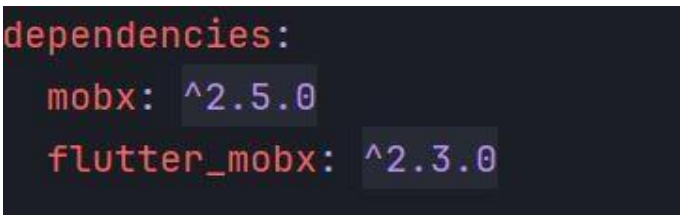


Рисунок 15 — Dependencies в файле pubspec.yaml для работы с MobX

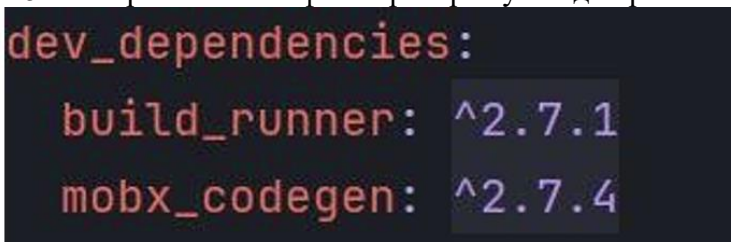


Рисунок 16 — Dev_dependencies в файле pubspec.yaml для работы с MobX

4.2 Реализация Store

Первое, что будет рассмотрено, это класс-делегат в MobX – Store. Store в MobX – это способ собрать связанное наблюдаемое состояние в одном классе. Store позволяет нам использовать аннотации и сохраняет код простым. Создадим новый файл lib/counter.dart и добавим в него содержимое с рисунка 17.



Рисунок 17 — Файл counter.dart

Абстрактный класс _Counter, включающий в себя примесь Store необходим нам для реализации всего кода класса-делегата. В то время как класс Counter, создаваемый выше используется для объединения класса делегата с генерируемым кодом MobX Store при помощи build_runner. Сгенерированный код будет находиться внутри файла counter.g.dart, который мы включаем в файл counter.dart при помощи ключевого слова part. Важно

заметить, что при отсутствии подключения сгенерационного файла в файл с классом-делегатом, генерация миксина _$Counter не пройдет успешно. Для указания данных, которые будут передаваться на отображения в слой представления, используется аннотация @observable. Если какое-либо поле помечено данной аннотацией, то помеченный объект становится наблюдаемый. Для обновления наблюдаемых данных используются методы, помеченные специальной аннотацией – @action. В данном примере она необходима для обозначения метода increment() как метода обновляющего данные класса-делегата.

Для генерации необходимых данных для MobX Store требуется прописать команду «flutter pub run build_runner build» в консоли или терминале.

4.3 Подключение Store к Observer

Когда реализация класса-делегата завершена, можно настроить наблюдателя на экране приложения. Для этого требуется произвести изменения в стартовом файле main.dart. Содержимое файла main.dart должно быть эквивалентно рисунку 18.



```dart
import 'package:flutter/material.dart';
import 'package:flutter_mobx/flutter_mobx.dart';
import 'counter.dart';

void main() {
  runApp(CounterApp());
}

class CounterApp extends StatelessWidget{
  const CounterApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'MobX Counter',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ), ThemeData
      home: CounterPage(),
    ); MaterialApp
  }
}
class CounterPage extends StatelessWidget {
  final counter = Counter();
  CounterPage({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('MobX Counter'),
      ), AppBar
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Text('You have pushed the button this many times:'),
            Observer(
              builder: (_) => Text(
                '${counter.value}',
                style: Theme.of(context).textTheme.headlineMedium,
              ), Text
            ), Observer
          ], <Widget>[]
        ), Column
      ), Center
      floatingActionButton: FloatingActionButton(
        onPressed: counter.increment,
        tooltip: 'Increment',
        child: Icon(Icons.add),
      ), FloatingActionButton
    ); Scaffold
  }
}
```

Рисунок 18 — Файл main.dart

В качестве основных изменений необходимо создать объект класса-делегата MobX Store, а также обернуть Text Widget с информацией о количестве нажатий в Widget наблюдателя – Observer. Конструктор виджета содержит: super.key — (опциональный) параметр ключа, builder —функция

которая возвращает виджет, определяет, что именно Observer будет наблюдать и отображать, name — (опционально) имя наблюдателя, может использоваться для отладки или идентификации, warnWhenNoObservables — (опционально) флаг, указывающий, следует ли выдавать предупреждение, если в наблюдении нет наблюдаемых объектов, как показано на рисунке 19.

```
class Observer extends StatelessObserverWidget {
  // ignore: prefer_const_constructors_in_immutables
  Observer({
    super.key,
    required this.builder,
    super.name,
    super.warnWhenNoObservables,
  }) : debugConstructingStackFrame = debugFindConstructingStackFrame();
```

Рисунок 19 — Виджет Observer

Далее остается только подставить значение, хранимое в классе-делегате, в Text Widgetи поменять метод обработки нажатия на FloatingActionButton с внутренней логики Stateful Widget-а на метод в классе-делегате. Так как более внутренняя логика, а соответственно и State у Stateful Widget-а более не нужны, то можно Stateful Widget переделать в Stateless Widget, для уменьшения занимаемой памяти.

5. Выполнение практической работы №9

Для переработки имеющегося приложения, выполненного на предыдущих практических работах, таким образом, чтобы в нем реализовывался один из рассмотренных в теоретической части способов хранения состояния с использованием MobX.

Для реализации работы с MobX в проект требуется в файл pubspec.yaml добавить пакеты mobx версии 2.5.0, flutter_mobx версии 2.3.0, build_runner версии 2.7.1 и mobx_codegen версии 2.7.4, как показано на рисунках 20–21. Данные версии были выбраны, так как они являются самыми актуальными на момент выполнения данной практической работы.

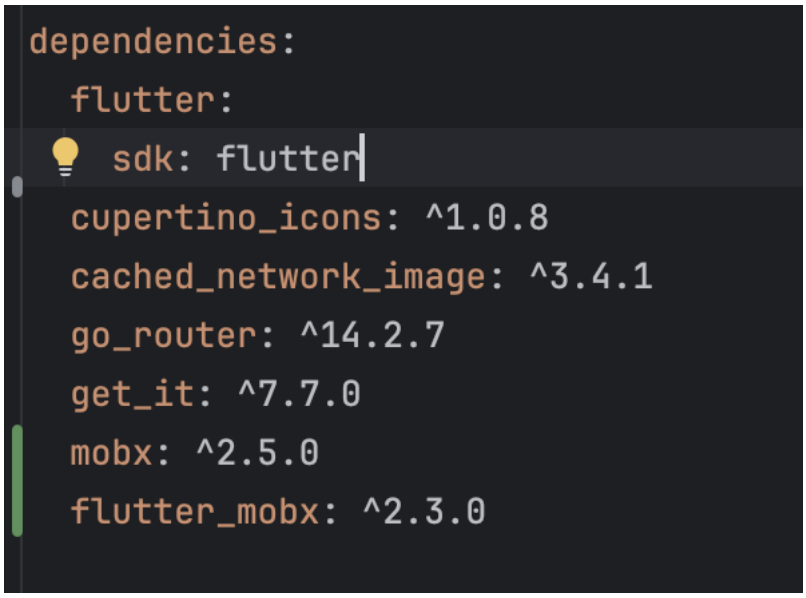


Рисунок 20 — Dependencies в файле pubspec.yaml для работы с MobX

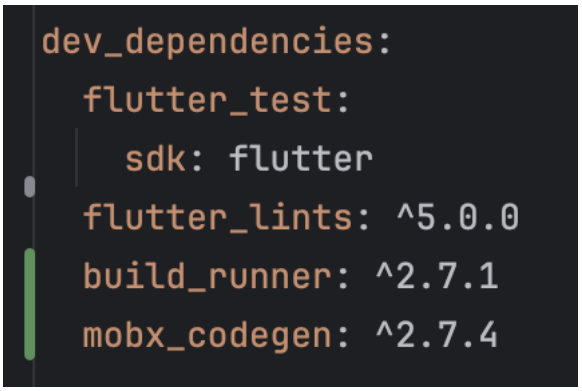
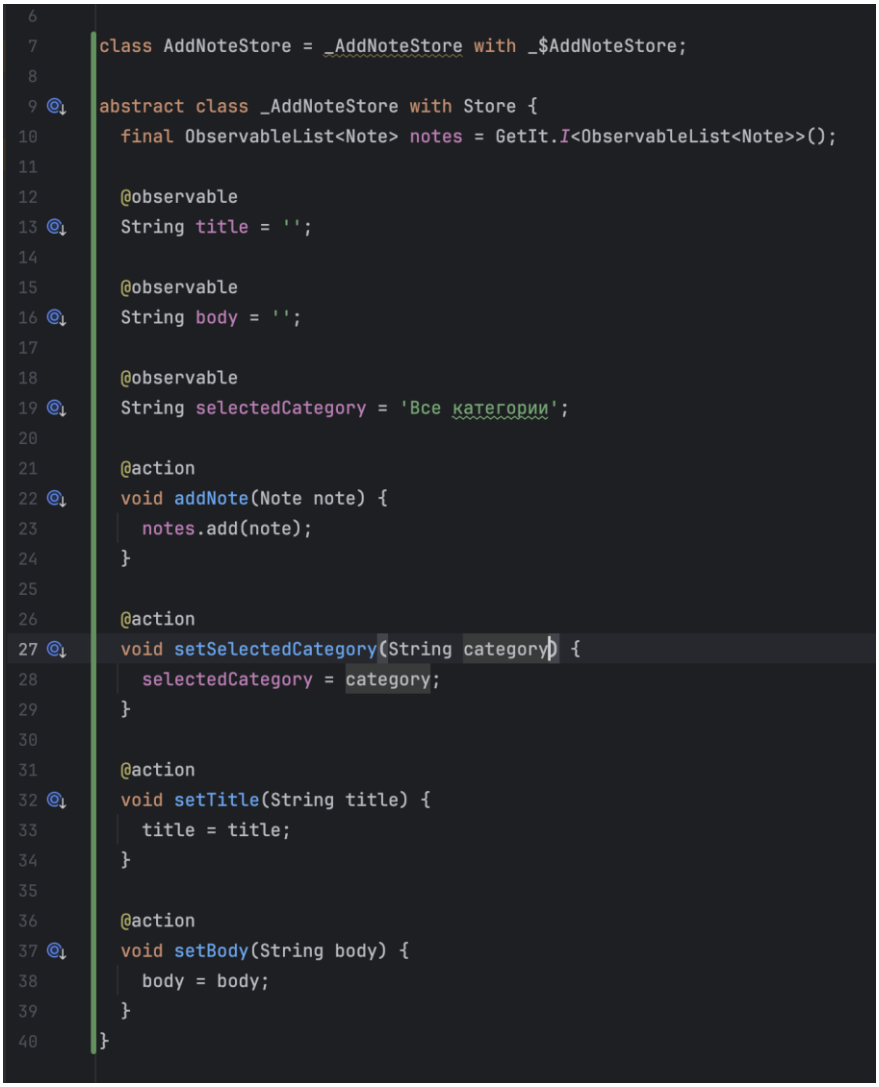


Рисунок 21 — Dev_dependencies в файле pubspec.yaml для работы с MobX

Далее были созданы отдельные классы-делегаты в MobX для каждого экрана: AddNoteStore для экрана добавления заметки, EditNoteStore для экрана редактирования заметки, NotesListStore для экрана списка заметок,

ArchiveStore для экрана архива и FavoritesStore для экрана избранного. Каждый store собирает связанное наблюдаемое состояние для своего экрана, включая общий ObservableList из GetIt, а также операции с ним, как показано на рисункках 22-26. Store позволяет нам использовать аннотации и сохраняет код простым.



```
class AddNoteStore = _AddNoteStore with _$AddNoteStore;

abstract class _AddNoteStore with Store {
  final ObservableList<Note> notes = GetIt.I<ObservableList<Note>>();

  @observable
  String title = '';

  @observable
  String body = '';

  @observable
  String selectedCategory = 'Все категории';

  @action
  void addNote(Note note) {
    notes.add(note);
  }

  @action
  void setSelectedCategory(String category) {
    selectedCategory = category;
  }

  @action
  void setTitle(String title) {
    title = title;
  }

  @action
  void setBody(String body) {
    body = body;
  }
}
```

Рисунок 22 — Класс AddNoteStore

```dart
class EditNoteStore = _EditNoteStore with _$EditNoteStore;

abstract class _EditNoteStore with Store {
  final ObservableList<Note> notes = GetIt.I<ObservableList<Note>>();

  @observable
  String title = '';

  @observable
  String body = '';

  @observable
  String selectedCategory = 'Все категории';

  @action
  void updateNote(String id, Note updatedNote) {
    final index = notes.indexWhere((note) => note.id == id);
    if (index != -1) {
      notes[index].title = updatedNote.title;
      notes[index].content = updatedNote.content;
      notes[index].category = updatedNote.category;
    }
  }

  @action
  void setSelectedCategory(String category) {
    selectedCategory = category;
  }

  @action
  void setTitle(String title) {
    title = title;
  }

  @action
  void setBody(String body) {
    body = body;
  }
}
```

Рисунок 23 - Класс EditNoteStore

```dart
abstract class _ArchiveStore with Store {
  final ObservableList<Note> notes = GetIt.I<ObservableList<Note>>();

  @observable
  String searchQuery = '';

  @observable
  String selectedCategory = 'Все категории';

  @observable
  String sortCriteria = 'Дата создания';

  @computed
  List<Note> get filteredNotes {
    final query = searchQuery.toLowerCase();
    var list = notes.where((note) {
      final matchesQuery = note.title.toLowerCase().contains(query) ||
          note.content.toLowerCase().contains(query);
      final matchesCategory = selectedCategory == 'Все категории' ||
          note.category == selectedCategory;
      return matchesQuery && matchesCategory && note.isArchived;
    }).toList();
    list.sort((a, b) {
      switch (sortCriteria) {
        case 'Заголовок':
          return a.title.compareTo(b.title);
        case 'Дата создания':
          return b.creationDate.compareTo(a.creationDate);
        default:
          return 0;
      }
    });
    return list;
  }
  @action
  void setSearchQuery(String query) {
    searchQuery = query;
  }
  @action
  void setSelectedCategory(String category) {
    selectedCategory = category;
  }
  @action
  void setSortCriteria(String criteria) {
    sortCriteria = criteria;
  }
  @action
  void deleteNote(String id) {
    notes.removeWhere((note) => note.id == id);
  }
  @action
  void toggleFavorite(String id) {
    final index = notes.indexWhere((note) => note.id == id);
```

Рисунок 24 - Класс ArchiveStore

```dart
class FavoritesStore = _FavoritesStore with _$FavoritesStore;
abstract class _FavoritesStore with Store {
  final ObservableList<Note> notes = GetIt.I<ObservableList<Note>>();
  @observable
  String searchQuery = '';
  @observable
  String selectedCategory = 'Все категории';
  @observable
  String sortCriteria = 'Дата создания';

  @computed
  List<Note> get filteredNotes {
    final query = searchQuery.toLowerCase();
    var list = notes.where((note) {
      final matchesQuery = note.title.toLowerCase().contains(query) ||
          note.content.toLowerCase().contains(query);
      final matchesCategory = selectedCategory == 'Все категории' ||
          note.category == selectedCategory;
      return matchesQuery && matchesCategory && note.isFavorite && !note.isArchived;
    }).toList();
    list.sort((a, b) {
      switch (sortCriteria) {
        case 'Заголовок':
          return a.title.compareTo(b.title);
        case 'Дата создания':
          return b.creationDate.compareTo(a.creationDate);
        default:
          return 0;
      }
    });
    return list;
  }
  @action
  void setSearchQuery(String query) {
    searchQuery = query;
  }
  @action
  void setSelectedCategory(String category) {
    selectedCategory = category;
  }
  @action
  void setSortCriteria(String criteria) {
    sortCriteria = criteria;
  }
  @action
  void deleteNote(String id) {
    notes.removeWhere((note) => note.id == id);
  }
  @action
  void toggleFavorite(String id) {
    final index = notes.indexWhere((note) => note.id == id);
    if (index != -1) {
      notes[index].isFavorite = !notes[index].isFavorite;
```

Рисунок 25 - Класс FavoritesStore



```
class FavoritesStore = _FavoritesStore with _$FavoritesStore;
abstract class _FavoritesStore with Store {
  final ObservableList<Note> notes = GetIt.I<ObservableList<Note>>();
  @observable
  String searchQuery = '';
  @observable
  String selectedCategory = 'Все категории';
  @observable
  String sortCriteria = 'Дата создания';

  @computed
  List<Note> get filteredNotes {
    final query = searchQuery.toLowerCase();
    var list = notes.where((note) {
      final matchesQuery = note.title.toLowerCase().contains(query) ||
          note.content.toLowerCase().contains(query);
      final matchesCategory = selectedCategory == 'Все категории' ||
          note.category == selectedCategory;
      return matchesQuery && matchesCategory && note.isFavorite && !note.isArchived;
    }).toList();
    list.sort((a, b) {
      switch (sortCriteria) {
        case 'Заголовок':
          return a.title.compareTo(b.title);
        case 'Дата создания':
          return b.creationDate.compareTo(a.creationDate);
        default:
          return 0;
      }
    });
    return list;
  }
  @action
  void setSearchQuery(String query) {
    searchQuery = query;
  }
  @action
  void setSelectedCategory(String category) {
    selectedCategory = category;
  }
  @action
  void setSortCriteria(String criteria) {
    sortCriteria = criteria;
  }
  @action
  void deleteNote(String id) {
    notes.removeWhere((note) => note.id == id);
  }
  @action
  void toggleFavorite(String id) {
    final index = notes.indexWhere((note) => note.id == id);
    if (index != -1) {
      notes[index].isFavorite = !notes[index].isFavorite;
```

Рисунок 26 - Класс NotesListStore

Далее для генерации необходимых данных для MobX Store требуется прописать команду «flutter pub run build_runner build» в терминале, как показано на рисунке 27.

```
vostreex@MacBook-Pro-Vasilij prack_9 % flutter pub run build_runner build
Deprecated. Use `dart run` instead.
6s mobx_codegen:mobx_generator on 25 inputs: 15 skipped, 3 same, 7 no-op; spent 4s analyzing, 1s resolving
0s source_gen:combining_builder on 25 inputs: 22 skipped, 3 same

Built with build_runner/jit in 6s; wrote 6 outputs.
```

Рисунок 27 — Запуск команды

После выполнения данной команды создаются файлы add_note_store.g.dart, edit_note_store.g.dart, notes_list_store.g.dart, archive_store.g.dart и favorites_store.g.dart (Рисунок 28).

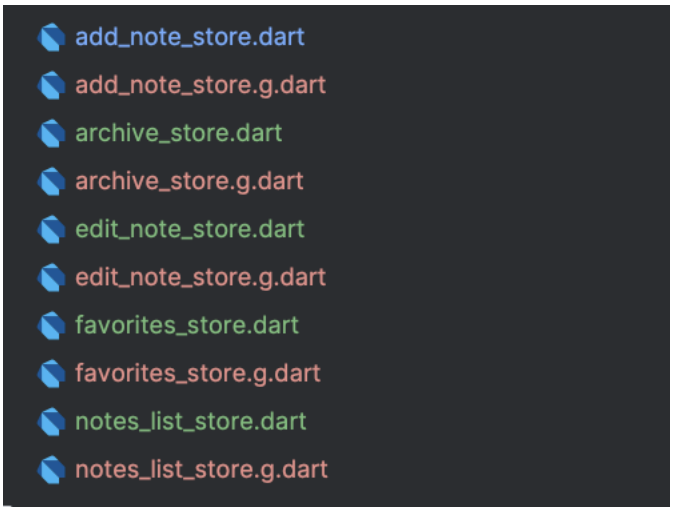


Рисунок 28 – Сгенерированные файлы

После создания классов, добавим регистрацию общего ObservableList в функцию main и зарегистрируем его в контейнере при помощи GetIt.I.registerSingleton(ObservableList()), как показано на рисунке 29.

```dart
void main() {
  GetIt.I.registerSingleton<ObservableList<Note>>(ObservableList<Note>());
  GetIt.I.registerSingleton<AddNoteStore>(AddNoteStore());
  GetIt.I.registerSingleton<EditNoteStore>(EditNoteStore());
  GetIt.I.registerSingleton<NotesListStore>(NotesListStore());
  GetIt.I.registerSingleton<ArchiveStore>(ArchiveStore());
  GetIt.I.registerSingleton<FavoritesStore>(FavoritesStore());
  runApp(MyApp());
}
```

Рисунок 29 — Регистрация общего состояния

Далее внедрим AddNoteStore в экране добавления заметки (AddNoteScreen). Для этого инициализируем поле store в методе initState, чтобы обеспечить безопасность относительно контекста, используя специализированный метод GetIt.I.get(), а также заменим вызовы репозитория

на вызовы к классу состояния и добавим Observer, как показано на рисунках 30–31.

```dart
class _AddNoteScreenState extends State<AddNoteScreen> {
  final TextEditingController _titleController = TextEditingController();
  final TextEditingController _contentController = TextEditingController();
  late AddNoteStore _store;

  @override
  void initState() {
    super.initState();
    _store = GetIt.I<AddNoteStore>();
    _titleController.addListener(() => _store.setTitle(_titleController.text));
    _contentController.addListener(() => _store.setTitle(_contentController.text));
  }
```

Рисунок 30 — Внедрение состояния через DI

```dart
Observer(
  builder: (_) => CategoryDropdown(
    value: _store.selectedCategory,
    onChanged: (String? newValue) {
      _store.setSelectedCategory(newValue!);
    },
  ), CategoryDropdown
), Observer
const SizedBox(height: 24.0),
ElevatedButton(
  onPressed: () {
    final title = _titleController.text.trim();
    final content = _contentController.text.trim();
    if (title.isNotEmpty && content.isNotEmpty) {
      _store.addNote(
        Note(
          title: title,
          content: content,
          category: _store.selectedCategory,
        ), Note
      );
      _titleController.clear();
      _contentController.clear();
      context.pop('/notes');
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Заполните оба поля')),
      );
    }
  },
```

Рисунок 31 — Обращения к классу состояния

Далее аналогично внедрим EditNoteStore в экран редактирования заметки (EditNoteScreen), также инициализировав поле store в методе initState, чтобы обеспечить безопасность относительно контекста, используя

специализированный метод GetIt.I.get<NoteStore>() как показано на рисунке 32, а также заменим вызовы репозитория на вызовы к классу состояния и добавим Observer, как показано на рисунке 33.

```dart
class _EditNoteScreenState extends State<EditNoteScreen> {
  final TextEditingController _titleController = TextEditingController();
  final TextEditingController _contentController = TextEditingController();
  late EditNoteStore _store;

  @override
  void initState() {
    super.initState();
    _store = GetIt.I<EditNoteStore>();
    _titleController.text = widget.note.title;
    _contentController.text = widget.note.content;
    _store.setTitle(widget.note.title);
    _store.setBody(widget.note.content);
    _store.setSelectedCategory(widget.note.category);
    _titleController.addListener(() {
      _store.setTitle(_titleController.text);
      setState(() {});
    });
    _contentController.addListener(() => _store.setBody(_contentController.text));
  }
```

Рисунок 32 — Внедрение класса состояния в EditNoteScreen

```dart
Observer(
  builder: (_) => CategoryDropdown(
    value: _store.selectedCategory,
    onChanged: (String? newValue) {
      _store.setSelectedCategory(newValue!);
    },
  ), CategoryDropdown
), Observer
const SizedBox(height: 24.0),
ElevatedButton(
  onPressed: () {
    final title = _titleController.text.trim();
    final content = _contentController.text.trim();
    if (title.isNotEmpty && content.isNotEmpty) {
      _store.updateNote(
        widget.index,
        Note(
          title: title,
          content: content,
          category: _store.selectedCategory,
          isFavorite: widget.note.isFavorite,
          isArchived: widget.note.isArchived,
        ), Note
      );
      context.pop();
    } else {
```

Рисунок 33— Обращения к классу состояния

Далее внедрим NotesListState в экран списка заметок (NotesListScreen), инициализировав поле store в методе initState, чтобы обеспечить безопасность

относительно контекста, используя специализированный метод GetIt.I.get<NoteStore>() как показано на рисунке 34, а также настроим наблюдателя на экране используя Widget наблюдателя – Observer, оборачивая ListView для реактивных обновлений списка заметок, как показано на рисунке 35. Также заменим вызовы репозитория на вызовы к классу состояния (Рисунок 36).

```dart
class _NotesListScreenState extends State<NotesListScreen> {
  final TextEditingController _controller = TextEditingController();
  late NotesListStore _store;

  @override
  void initState() {
    super.initState();
    _store = GetIt.I<NotesListStore>();
    _controller.addListener(() => _store.setSearchQuery(_controller.text));
  }
```

Рисунок 34 — Внедрение класса состояния в NotesListScreen

```dart
        const SizedBox(width: 16.0),
        Observer(
          builder: (_) => CategoryDropdown(
            value: _store.selectedCategory,
            onChanged: (String? newValue) {
              _store.setSelectedCategory(newValue!);
            },
          ),
        ),
      ],
    ),
    const SizedBox(height: 20.0),
    Observer(
      builder: (_) => Container(
        padding: const EdgeInsets.symmetric(horizontal: 12.0, vertical: 8.0),
        decoration: BoxDecoration(
          border: Border.all(color: Colors.grey[400]!),
          borderRadius: BorderRadius.circular(12.0),
          color: Colors.grey[100],
        ),
        child: Row(
          children: [
            Text(
              'Сортировать по:',
              style: TextStyle(
```

Рисунок 35 — Добавление Observer в NotesListScreen



```
void _deleteNote(String id) {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Подтверждение'),
      content: const Text('Вы уверены, что хотите удалить эту заметку?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(context).pop(),
          child: const Text('Отмена'),
        ), TextButton
        TextButton(
          onPressed: () {
            _store.deleteNote(id);
            Navigator.of(context).pop();
          },
          child: const Text('Удалить'),
        ), TextButton
      ],
    ), AlertDialog
  );
}

void _toggleFavorite(BuildContext ctx, String id) {
  final note = _store.notes.firstWhere((n) => n.id == id);
  final wasFavorite = note.isFavorite;
  _store.toggleFavorite(id);
  ScaffoldMessenger.of(ctx).showSnackBar(
    SnackBar(
      content: Text(
        wasFavorite ? 'Заметка удалена из избранного' : 'Заметка добавлена в избранное',
      ), Text
    ), SnackBar
  );
}

void _toggleArchive(BuildContext ctx, String id) {
  final note = _store.notes.firstWhere((n) => n.id == id);
  final wasArchived = note.isArchived;
  _store.toggleArchive(id);
  ScaffoldMessenger.of(ctx).showSnackBar(
    SnackBar(
      content: Text(
        wasArchived ? 'Заметка восстановлена из архива' : 'Заметка добавлена в архив',
      ), Text
    ), SnackBar
  );
}
```

Рисунок 36 — Обращения к классу состояния

Аналогично внедрим ArchiveStore в экран архива (ArchiveScreen), инициализировав поле store в методе initState, заменив вызовы репозитория на store, и оборачивая ListView в Observer для реактивных обновлений, как показано на рисунках 37–38. Также заменим вызовы репозитория на вызовы к классу состояния (Рисунок 39).

```dart
class ArchiveScreen extends StatefulWidget {
  const ArchiveScreen({super.key});

  @override
  _ArchiveScreenState createState() => _ArchiveScreenState();
}

class _ArchiveScreenState extends State<ArchiveScreen> {
  final TextEditingController _controller = TextEditingController();
  late ArchiveStore _store;

  @override
  void initState() {
    super.initState();
    _store = GetIt.I<ArchiveStore>();
    _controller.addListener(() => _store.setSearchQuery(_controller.text));
  }
```

Рисунок 37 — Внедрение класса состояния в ArchiveScreen

```dart
        const SizedBox(width: 16.0),
        Observer(
          builder: (_) => CategoryDropdown(
            value: _store.selectedCategory,
            onChanged: (String? newValue) {
              _store.setSelectedCategory(newValue!);
            },
          ), CategoryDropdown
        ), Observer
      ],
    ), Row
    const SizedBox(height: 20.0),
    Observer(
      builder: (_) => Container(
        padding: const EdgeInsets.symmetric(horizontal: 12.0, vertical: 8.0),
        decoration: BoxDecoration(
          border: Border.all(color: Colors.grey[400]!),
          borderRadius: BorderRadius.circular(12.0),
          color: Colors.grey[100],
        ), BoxDecoration
        child: Row(
          children: [
            Text(
              'Сортировать по:',
              style: TextStyle(
                fontSize: 14.0,
                color: Colors.grey[700],
              ), TextStyle
            ), Text
            const SizedBox(width: 8.0),
            Expanded(
              child: DropdownButtonHideUnderline(
```

Рисунок 38 — Добавление Observer в ArchiveScreen



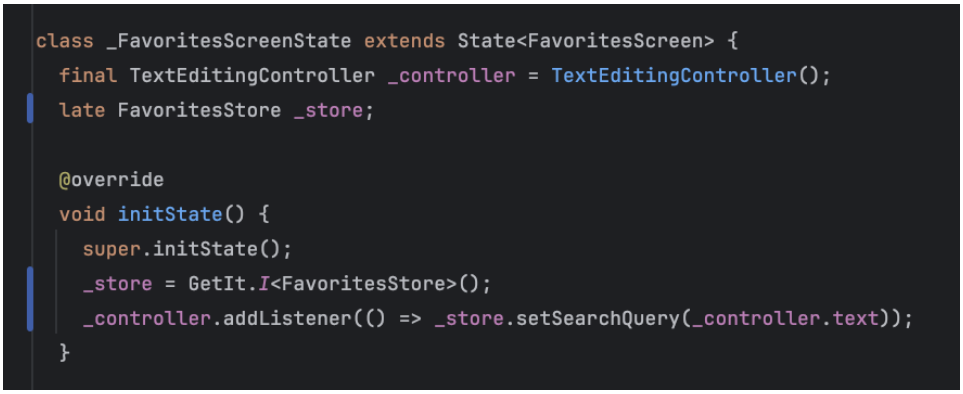
```
void _toggleFavorite(BuildContext ctx, String id) {
  final note = _store.notes.firstWhere((n) => n.id == id);
  final wasFavorite = note.isFavorite;
  _store.toggleFavorite(id);
  ScaffoldMessenger.of(ctx).showSnackBar(
    SnackBar(
      content: Text(
        wasFavorite ? 'Заметка удалена из избранного' : 'Заметка добавлена в избранное',
      ), Text
    ), SnackBar
  );
}

void _toggleArchive(BuildContext ctx, String id) {
  final note = _store.notes.firstWhere((n) => n.id == id);
  final wasArchived = note.isArchived;
  _store.toggleArchive(id);
  ScaffoldMessenger.of(ctx).showSnackBar(
    SnackBar(
      content: Text(
        wasArchived ? 'Заметка восстановлена из архива' : 'Заметка добавлена в архив',
      ), Text
    ), SnackBar
  );
}

void _deleteNote(String id) {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Подтверждение'),
      content: const Text('Вы уверены, что хотите удалить эту заметку?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(context).pop(),
          child: const Text('Отмена'),
        ), TextButton
        TextButton(
          onPressed: () {
            _store.deleteNote(id);
            Navigator.of(context).pop();
          },
          child: const Text('Удалить'),
        ), TextButton
      ],
    ), AlertDialog
  );
}
```

Рисунок 39 — Обращения к классу состояния

Далее внедрим NoteStore в экран избранного (FavoritesScreen), инициализировав поле store в методе initState, заменив вызовы репозитория на store, и оборачивая ListView в Observer для реактивных обновлений, как показано на рисунках 40–41. Также заменим вызовы репозитория на вызовы к классу состояния (Рисунок 42).



```
class _FavoritesScreenState extends State<FavoritesScreen> {
  final TextEditingController _controller = TextEditingController();
  late FavoritesStore _store;

  @override
  void initState() {
    super.initState();
    _store = GetIt.I<FavoritesStore>();
    _controller.addListener(() => _store.setSearchQuery(_controller.text));
  }
```

Рисунок 40 — Внедрение класса состояния в FavoritesScreen



Рисунок 41 — Добавление Observer в FavoritesScreen



Рисунок 42 — Обращения к классу состояния

Протестируем работу приложения с заметками на платформе MacOS.

После запуска приложения на экране будет изображен splash-экран, как показано на рисунке 43. После задержки перейдем к списку заметок.



Рисунок 43 — Splash-экран приложения

После этого откроется экран со списком заметок, как показано на рисунке 44.

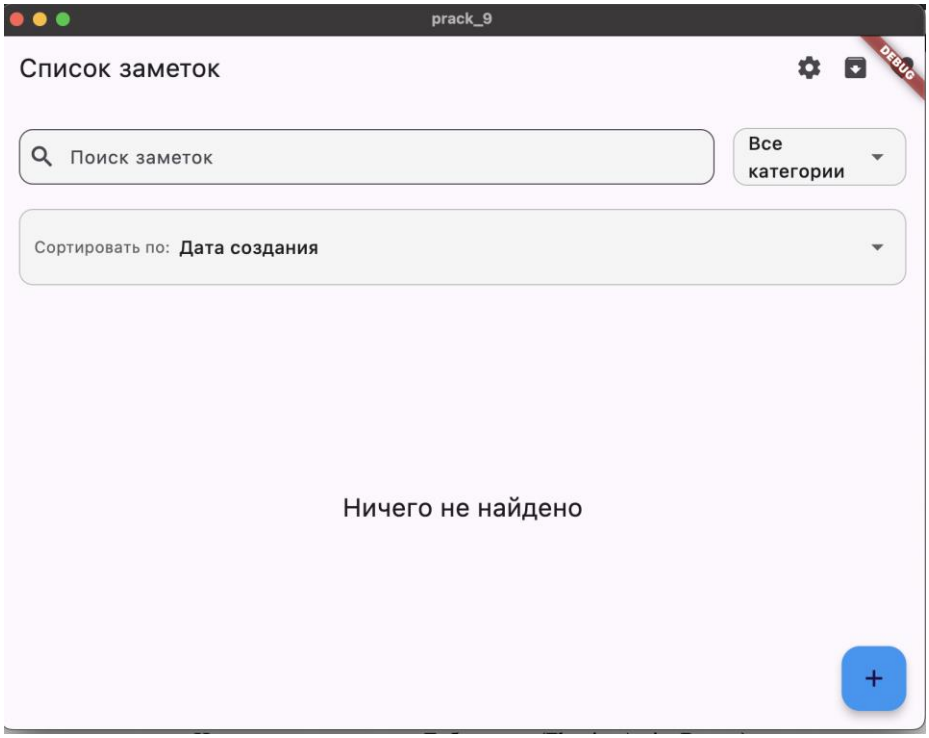


Рисунок 44 — Экран со списком заметок

Нажмем на кнопку «Добавить» (FloatingActionButton), после чего откроется экран добавления заметки, как показано на рисунке 45.



Рисунок 45 — Экран добавления заметки

Введем заголовок, содержимое и выберем категорию, после чего нажмем «Добавить» (Рисунок 46). Откроется экран со списком заметок, на котором можно увидеть добавленную заметку, как показано на рисунке 47.



Рисунок 46 — Заполнение формы создания заметки

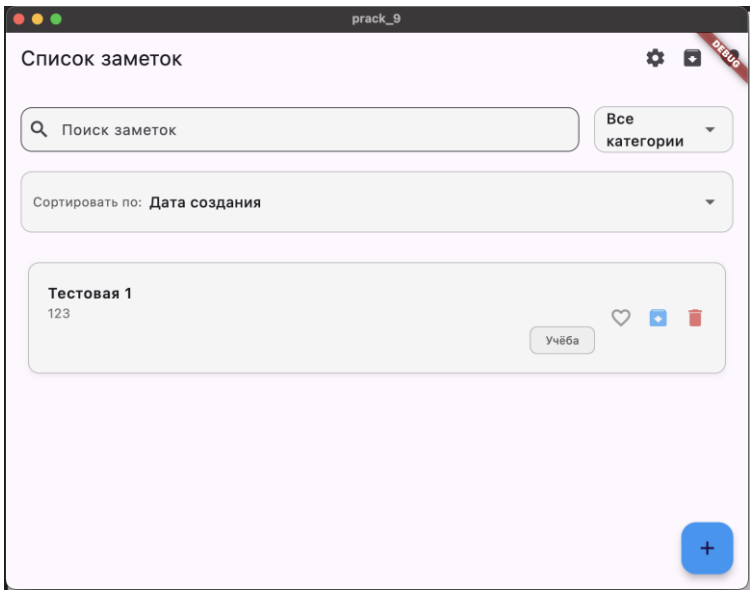


Рисунок 47 — Экран со списком заметок после добавления

Далее нажмем на заметку, после чего откроется экран редактирования заметки, где изменим содержимое, как показано на рисунке 48.

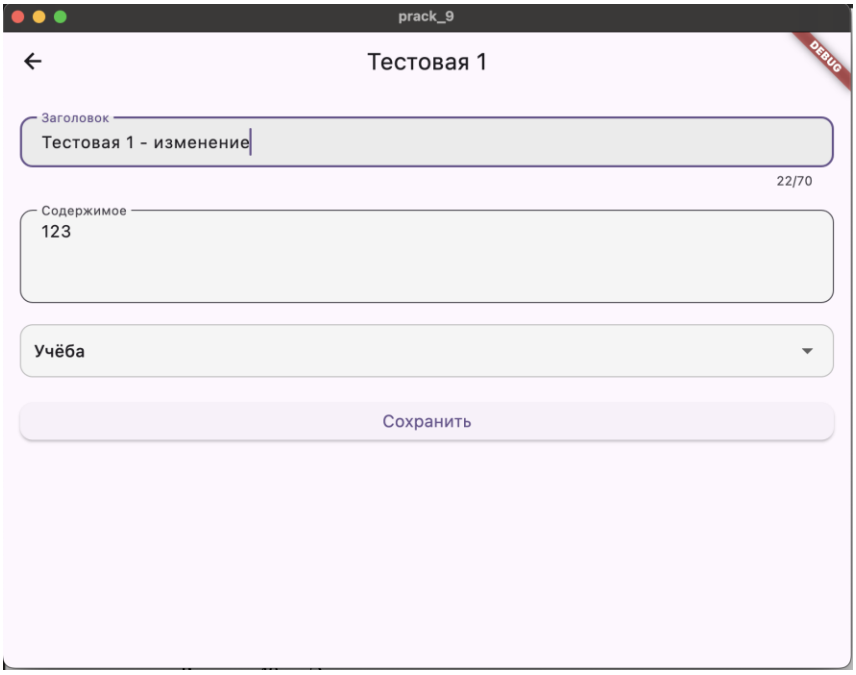


Рисунок 48 — Экран редактирования заметки

После нажатия на кнопку «Сохранить» откроется экран со списком заметок, на котором можно увидеть отредактированную заметку, как показано на рисунке 49.

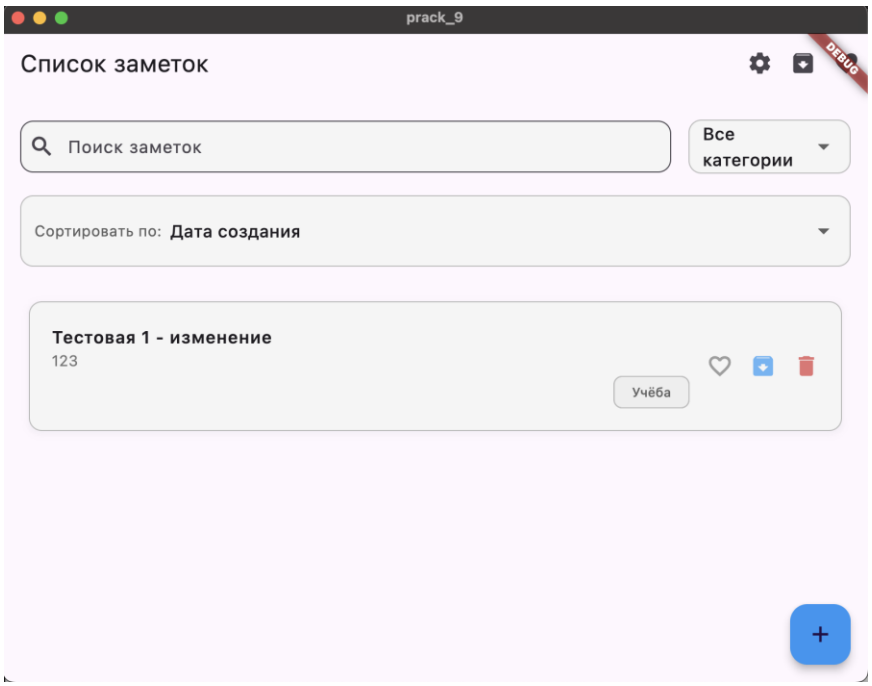


Рисунок 49 — Экран со списком заметок после редактирования

Далее добавим заметку в избранное нажав на соответствующую иконку у заметки (Рисунок 50–51).

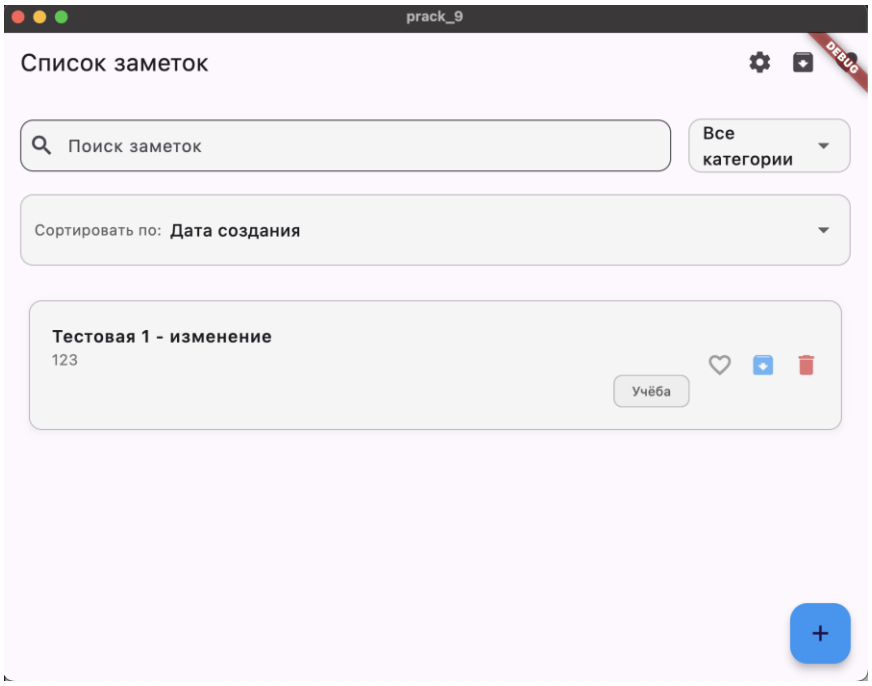


Рисунок 50 — Экран со списком заметок до добавления заметки в избранное

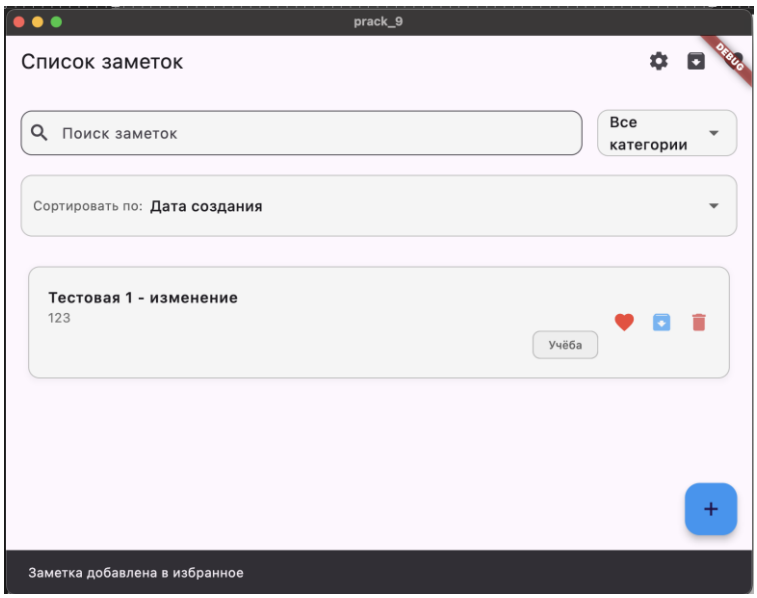


Рисунок 51 — Экран со списком заметок после добавления заметки в избранное

Затем переключимся на экран избранного через иконку в AppBar и увидим добавленную заметку на экране (Рисунок 52).

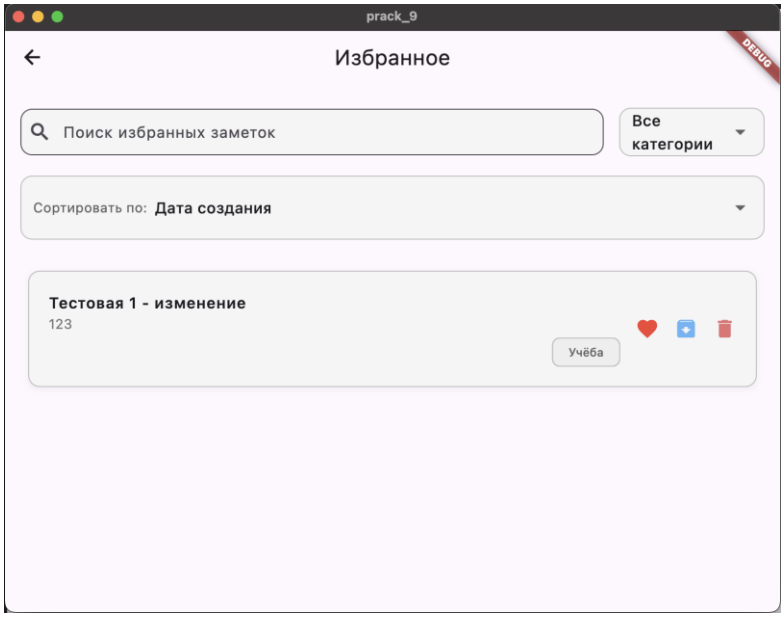


Рисунок 52— Добавленная заметка на экране избранного

Уберем заметку из избранного (toggleFavorite), и убедимся в обновлении списка, как показано на рисунке 53.

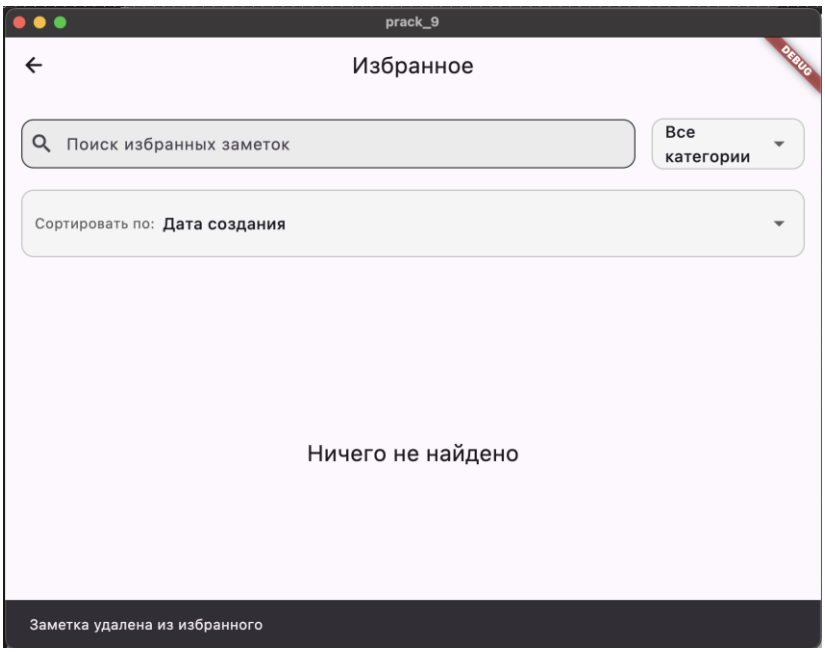


Рисунок 53 — Обновленный экран избранного без заметки

Аналогично добавим заметку в архив из экрана списка заметок нажав на соответствующую иконку у заметки (Рисунок 54–55).

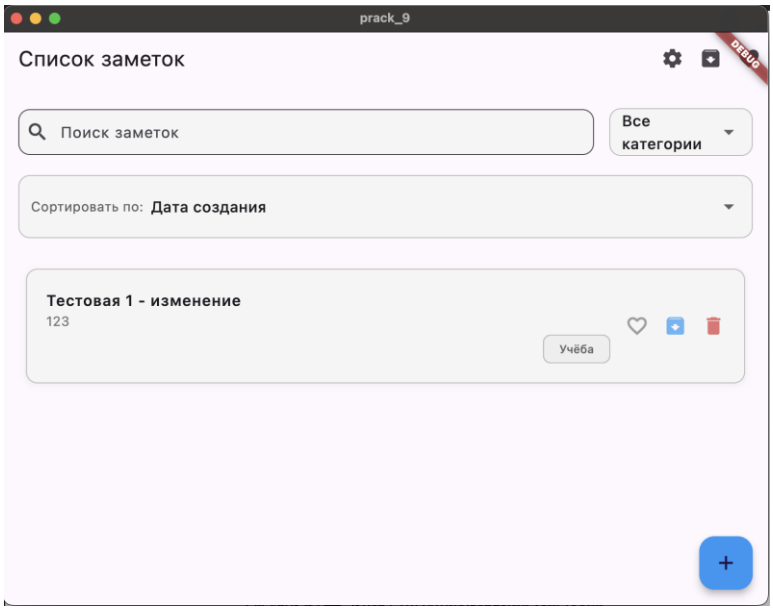


Рисунок 54 — Экран списка заметок до добавления заметки в архив

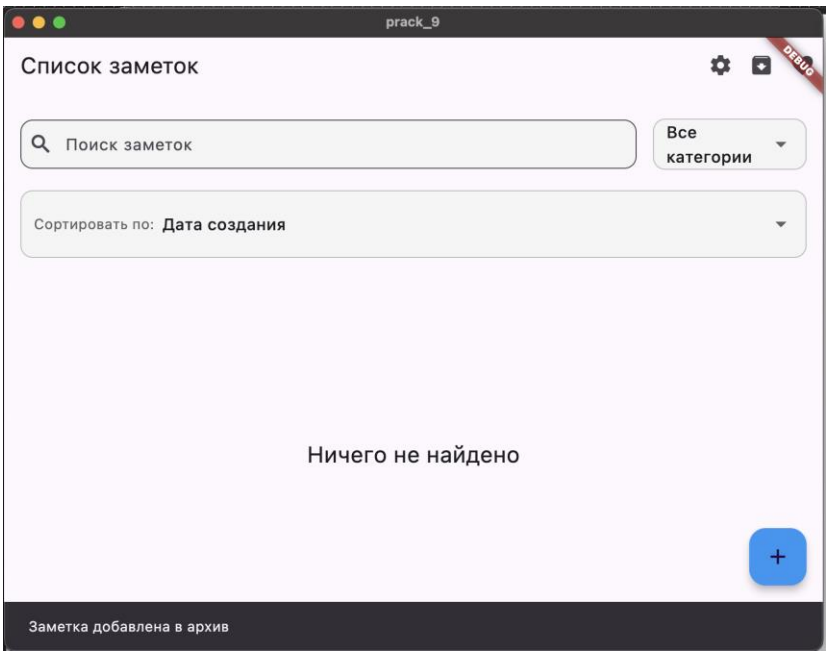


Рисунок 55 — Экран списка заметок после добавления заметки в архив

Затем переключимся на экран архива через иконку в AppBar увидим добавленную заметку на экране (Рисунок 56).

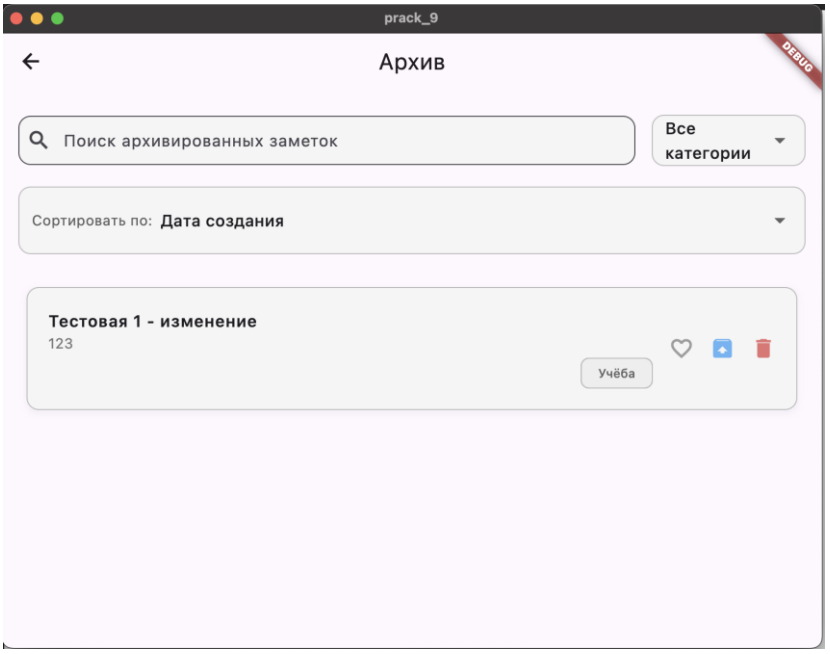


Рисунок 56 — Экран архива с заметкой

Уберем заметку из архива (toggleArchive), и убедимся в обновлении списка, как показано на рисунке 57.

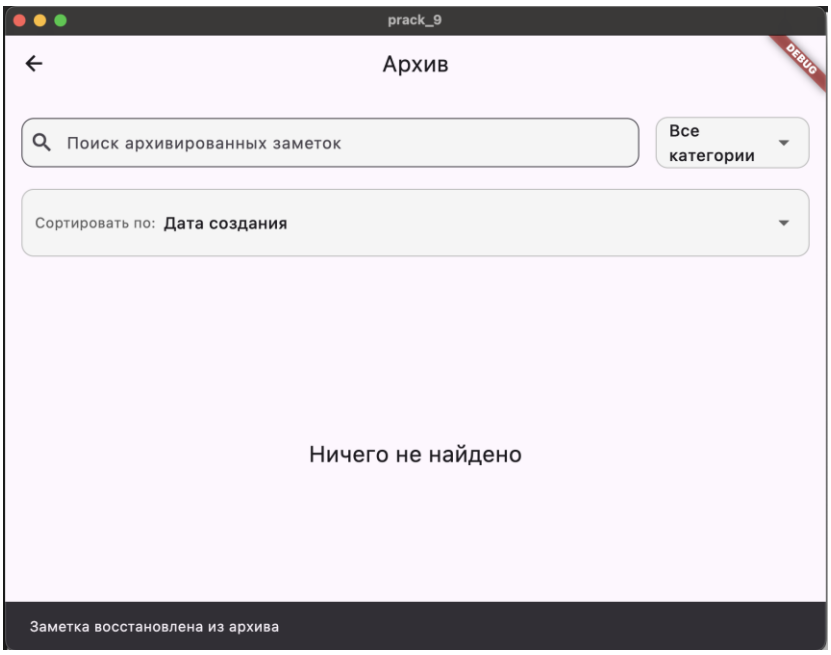


Рисунок 57— Обновленный экран архива без заметки

На экране списка заметок нажмем на дропдаун сортировки, после чего заметки отсортируются по выбранному критерию (Заголовок), как показано на рисунках 58–59.

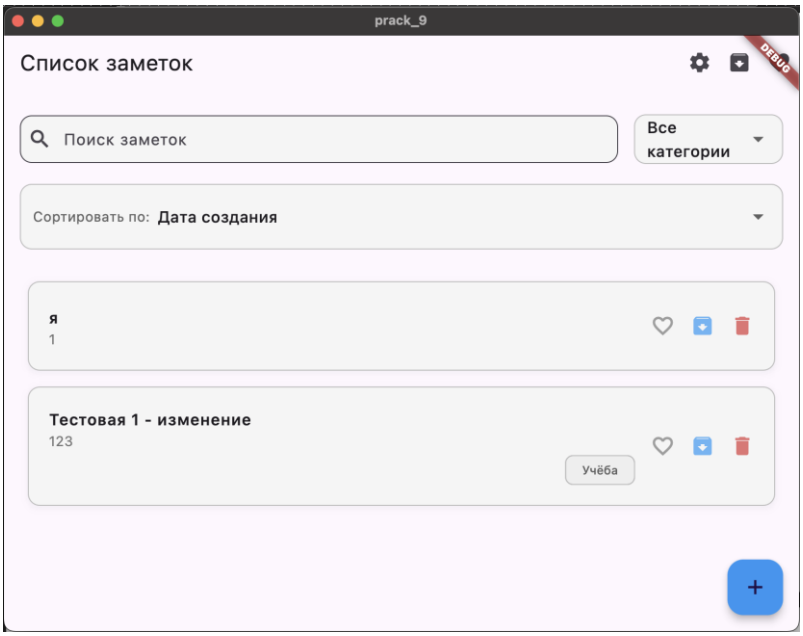


Рисунок 58 — Экран списка заметок до применения сортировки заметок

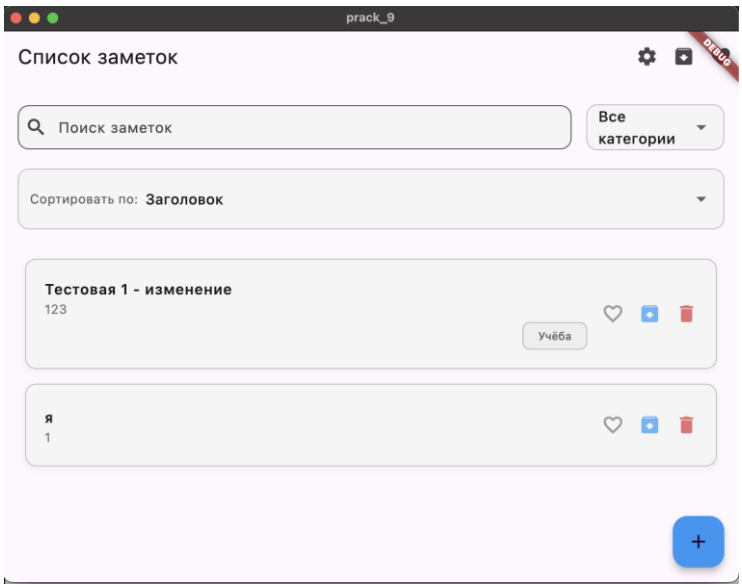


Рисунок 59 — Экран списка заметок после применения сортировки заметок

Далее на экране списка заметок нажмем на дропдаун фильтрации, после чего заметки от фильтруются по выбранному критерию (Учёба), как показано на рисунках 60–61.

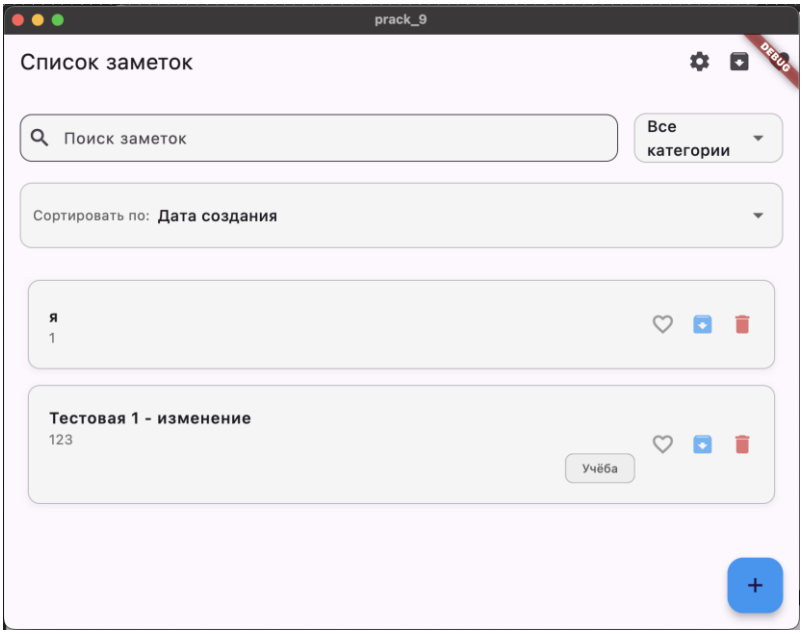


Рисунок 60 — Экран списка заметок до применения фильтрации заметок

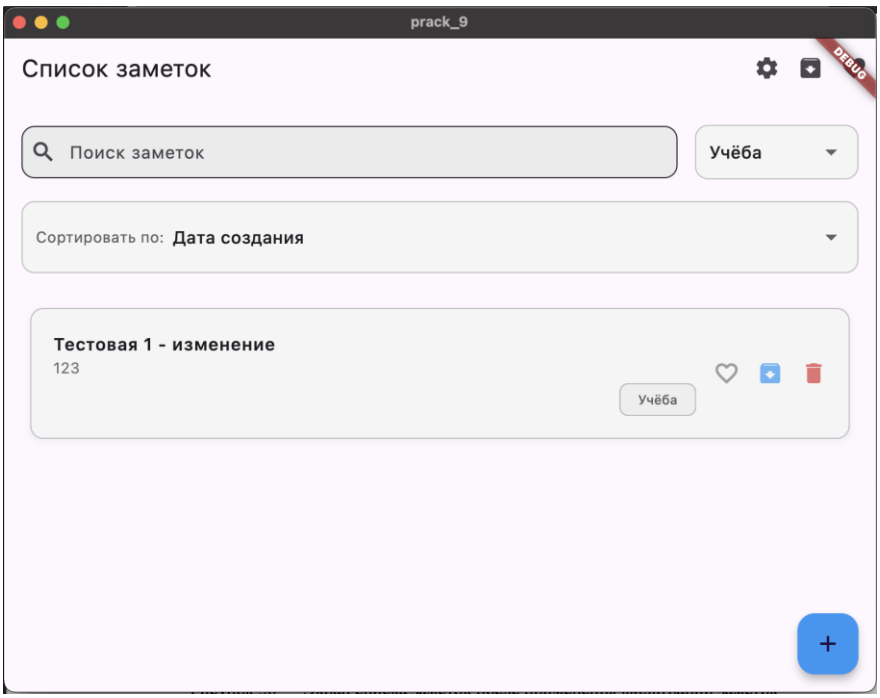


Рисунок 61 — Экран списка заметок после применения фильтрации заметок

Далее удалим заметку, нажав на иконку удаления, как показано на рисунке 62. После подтверждения заметка удалится, как показано на рисунке 63-64.

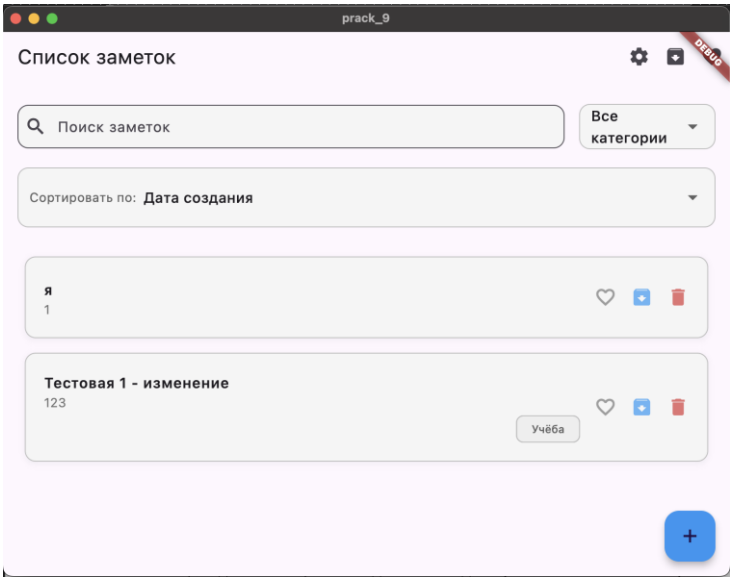


Рисунок 62 — Экран списка заметок до удаления заметки

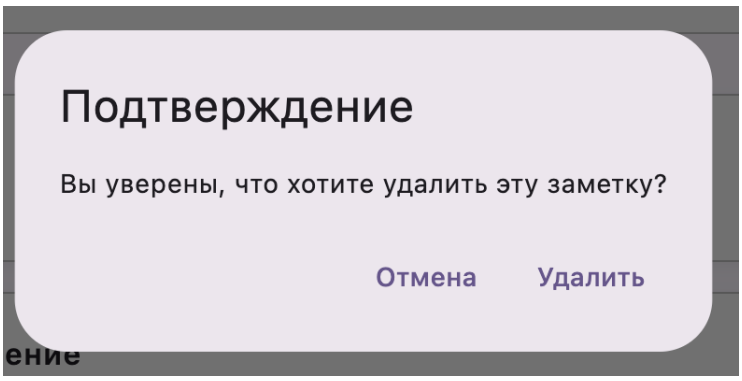


Рисунок 63 – Подтверждение удаления

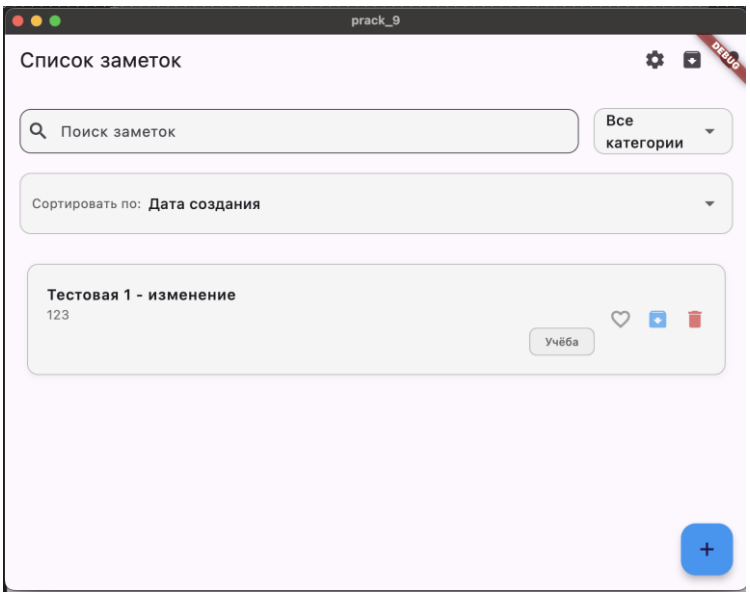


Рисунок 64 — Экран списка заметок после удаления заметки

Были протестированы все функции работы с заметками используя состояния, следовательно, можно сделать вывод что работа с состояниями при помощи MobX сделана верно.

Заключение

В процессе выполнения практической работы ознакомиться с задачами хранения состояния, рассмотрел способы хранения состояния и реализовал один из них при помощи MobX.

Исходный код выполненного задания представлен по ссылке: https://github.com/Andrew-Savin-msk/prac9